

Komorebi Café

Komorebi Café

Plataforma de gestión integral
para cadena de cafés temáticos

Documentación de Proyecto Final

Ciclo Formativo de Grado Superior —Desarrollo de Aplicaciones Web

Alumna: Irene Reyes Carmona

Centro: I.E.S. Ágora

Tutora: María Lourdes Calleja Flores

Año académico: 2025-2026

Resumen

Komorebi Café es una plataforma web de gestión integral diseñada para una cadena ficticia de catorce cafés temáticos japoneses. El nombre del proyecto proviene del término japonés *komorebi* (木漏れ日), que describe la luz solar filtrada entre las hojas de los árboles, evocando la atmósfera serena y cuidada que caracteriza estos espacios.

El sistema cubre la totalidad del ciclo operativo: gestión de reservas con control de idempotencia, carta digital con filtros por alérgeno, programa de fidelización, panel de cocina en tiempo real (Kitchen Display System (KDS)), control de salud de animales del local, operaciones de recepción con check-in/checkout y generación asíncrona de facturas en PDF.

Desde el punto de vista técnico, la solución está construida sobre PHP 8.4 con un framework Model-View-Controller (MVC) propio (sin Laravel ni Symfony como capa de aplicación), siguiendo los estándares PHP Standards Recommendation (PSR)-7, PSR-14 y PSR-15. La arquitectura sigue los principios 12-Factor App, con despliegue mediante Docker Compose y Railway. El sistema incluye más de 250 rutas, 53 servicios de negocio, 29 repositorios y aproximadamente 80 endpoints de Application Programming Interface (API) Representational State Transfer (REST).

Palabras clave: PHP 8.4, MVC, Docker, PSR-7, PSR-15, Result pattern, reservas, fidelización, cafés temáticos, 12-Factor.

Abstract

Komorebi Café is a comprehensive web management platform designed for a fictional chain of fourteen Japanese-themed cafés. The project name derives from the Japanese term *komorebi* (木漏れ日), describing sunlight filtering through tree leaves, evoking the serene and carefully crafted atmosphere that defines these spaces.

The system covers the complete operational cycle: reservation management with idempotency control, a digital menu with allergen filters, a loyalty programme, a real-time Kitchen Display System (KDS), animal health tracking, reception operations with check-in/checkout, and asynchronous PDF invoice generation.

Technically, the solution is built on PHP 8.4 with a custom MVC framework (no Laravel/Symfony application layer), following PSR-7, PSR-14 and PSR-15 standards. The architecture adheres to the 12-Factor App principles, deployed via Docker Compose and Railway. The system comprises over 250 routes, 53 business services, 29 repositories and approximately 80 REST API endpoints.

Keywords: PHP 8.4, MVC, Docker, PSR-7, PSR-15, Result pattern, reservations, loyalty, themed cafés, 12-Factor.

Agradecimientos

A María Lourdes Calleja Flores, tutora de este proyecto, por su orientación constante, su rigor técnico y su disponibilidad a lo largo de todo el proceso de desarrollo y documentación.

A mis compañeros y compañeras del ciclo, por los debates técnicos, las horas compartidas y el apoyo mutuo que han hecho posible completar este proyecto.

Y a todas las personas que, como la luz entre las hojas, iluminan los rincones del camino cuando más lo hace falta.

Índice general

Índice de figuras

Índice de tablas

Capítulo 1

Introducción General

1.1. Origen y concepto del proyecto

El punto de partida de Komorebi Café es la pregunta: ¿puede una aplicación web de gestión ser, al mismo tiempo, un reflejo fiel de la identidad de la marca que sirve? El proyecto nace de la voluntad de demostrar que la tecnología y la estética no son opuestos: un sistema de software robusto puede encarnar la filosofía tranquila, detallista y contemplativa que inspira el concepto de *komorebi*.

La palabra japonesa *komorebi* (木漏れ日) se compone de tres kanjis:

- 木 (*ki*) —árbol
- 漏れ (*more*) —filtrar, escapar
- 日 (*bi*) —luz, sol

Juntos describen ese fenómeno visual fugaz en que la luz solar se cuela entre las hojas creando patrones de claroscuro. Es una palabra sin traducción exacta al español, pero precisamente esa intraducibilidad la convierte en símbolo perfecto: algunas experiencias solo se pueden *vivir*, no describir.

1.2. Los catorce cafés temáticos

La cadena ficticia está compuesta por catorce cafés, cada uno inspirado en un universo estético japonés diferente. La cuadro 1.1 recoge los establecimientos registrados en el sistema.

Tabla 1.1: Catálogo de cafés temáticos de Komorebi Café

Slug	Nombre	Animal / Categoría
neko-no-niwa	Neko no Niwa (猫の庭)	Gato (lounge)
usagi-paradise	Usagi Paradise (うさぎパラダイス)	Conejo (lounge)
soft-cloud	Soft Cloud (ソフトクラウド)	Chinchilla (lounge)
chipmunk-forest	Chipmunk Forest (シマリスの森)	Ardilla (lounge)
mame-shiba-cafe	Mame Shiba Café (豆柴カフェ)	Perro Shiba Inu (playroom)
mipig-cafe	Mipig Cafe (マイピッグカフェ)	Cerdito miniatura (playroom)
parrot-talk	Parrot Talk (おしゃべりオウム)	Loro (playroom)
capyba-land	Capyba Land (カピバランド)	Capybara (farm)
alpaca-hill	Alpaca Hill (アルパカの丘)	Alpaca (farm)
little-hooves	Little Hooves (小さなひづめ)	Caballo miniatura (farm)

(continuación de la cuadro 1.1)

Slug	Nombre	Animal / Categoría
quack-club	Quack Club (クワッククラブ)	Pato (farm)
pui-pui-house	Pui Pui House (プイプイハウス)	Cobaya (zen)
prairie-town	Prairie Town (プレーリータウン)	Perrito de pradera (zen)
slow-life	Slow Life (スローライフ)	Tortuga (zen)

1.3. Misión, visión y valores

Misión

Ofrecer a cada cliente una experiencia de visita única, combinando una carta artesanal con un entorno temático cuidado y un servicio personalizado habilitado por tecnología.

Visión

Ser la referencia en España de la hostelería temática de inspiración japonesa, reconocidos tanto por la calidad de nuestros productos como por la excelencia tecnológica de nuestros sistemas de gestión.

Valores

Ma (間, el espacio entre las cosas), *omotenashi* (hospitalidad total), sostenibilidad, belleza en los detalles y apertura cultural.

1.4. Alcance y finalidad del proyecto

El proyecto abarca el diseño, implementación, pruebas y documentación de una plataforma web completa que gestiona:

1. La presencia pública de la cadena (carta, reservas en línea, historia, FAQ)
2. La autenticación y gestión de cuentas de usuario
3. El programa de fidelización con tarjeta digital y canjes
4. El backoffice de administración (usuarios, cafés, menú, reseñas, animales)
5. Los paneles de operación: recepción, cocina (KDS), manager y keeper
6. La API REST para integración con clientes móviles y terceros

1.5. Stack tecnológico

La cuadro 1.2 resume las tecnologías principales utilizadas en el proyecto.

Tabla 1.2: Stack tecnológico del proyecto

Capa	Tecnología	Versión / Detalles
Lenguaje backend	PHP	8.4 (strict types en todos los archivos)
Framework	MVC custom	Sin Laravel / Symfony, PSR-7/14/15
Servidor HTTP	FrankenPHP	Caddy + PHP integrado
Base de datos	MySQL	8.0, 19 migraciones SQL
Caché / Cola	Redis	7.x, sesiones + cola de trabajos
Frontend	Alpine.js	3.x, sin bundler en producción
CSS	CSS3 custom	Variables, Grid, Flexbox
Contenedores	Docker Compose	Multistage build
Despliegue	Railway	+ GitHub Actions Continuous Integration / Continuous Delivery
Tests unitarios	PHPUnit	11.x, tests paralelos
Tests E2E	Playwright	TypeScript
Análisis estático	PHPStan + Psalm	Nivel 5 + baseline
Email	Mailpit (dev)	SMTP local
Facturas PDF	Dompdf	Generación asíncrona

1.6. Metodología de desarrollo

El desarrollo siguió una metodología ágil inspirada en **Scrum**, adaptada al contexto de un equipo unipersonal. Se utilizó Trello como tablero Kanban con las columnas *Backlog*, *En progreso*, *Revisión* y *Hecho*.

El control de versiones se gestionó con Git siguiendo el flujo **Git Flow**:

- `main` —código en producción (Railway)
- `develop` —integración continua
- `feature/nombre` —desarrollo de funcionalidades
- `hotfix/nombre` —correcciones urgentes

Cada *pull request* requería superar el *quality gate* completo: PHPStan nivel 5, Psalm, suite PHPUnit y verificación de estilo PSR-12.

Capítulo 2

Desarrollo Técnico

2.1. Arquitectura del sistema

Komorebi Café implementa el patrón MVC desde cero, sin delegar en ningún framework de aplicación externo. El objetivo es demostrar comprensión profunda de los mecanismos subyacentes: enrutamiento, inyección de dependencias, ciclo de petición-respuesta y gestión de middleware.

2.1.1. Capas principales

public/index.php —Front controller

Único punto de entrada. Lee variables de entorno, inicializa el contenedor de dependencias y pasa la petición al router.

bootstrap/container.php —Contenedor IoC

Registra todos los *Service Providers*. Ciclo `register→boot`. Singletons enlazados por interfaz.

app/routes.php —Definición de rutas

250 rutas agrupadas por dominio con prefijos y middleware declarados de forma explícita.

app/Core/ —Framework interno

Router PSR-7/15, Contenedor IoC, Vista (auto-escape + Raw), Base de datos (PDO), Caché (Redis + local), Cola (Redis), Logger (stdout 12-Factor).

app/Http/Controllers/ —Controladores

Agrupados en: Admin/, Api/V1/, Auth/, Kitchen/, Keeper/, Manager/, Public/, Reception/, Shared/, Supervisor/, User/.

app/Services/ —Lógica de negocio

53 servicios. Todos devuelven `Result::ok()/fail()` —nunca lanzan excepciones para fallos esperados.

app/Repositories/ —Capa de datos

29 repositorios. Extienden `AbstractRepository` con PDO directo.

app/Events/ + Listeners/ —Eventos asíncronos

PSR-14 con Symfony `EventDispatcher`. Procesados por workers Redis.

migrations/ —Esquema de base de datos

19 archivos SQL numerados, aplicables mediante `make db-migrate`.

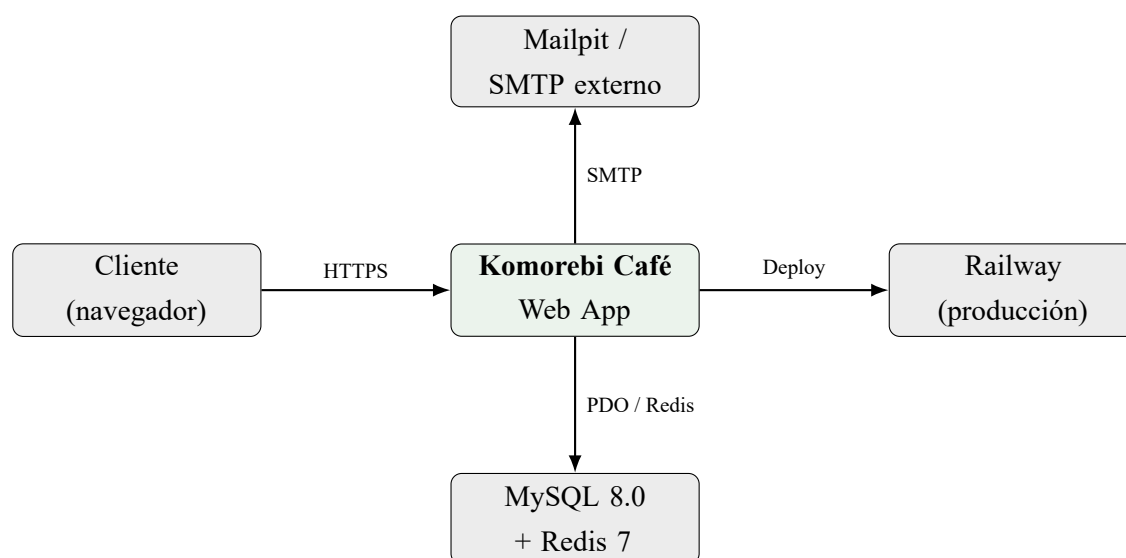
2.1.2. Diagrama de contexto C4 (nivel 1)

Figura 2.1: Diagrama de contexto C4 —nivel sistema

2.2. Ciclo de petición HTTP y pipeline de middleware

Cada petición HTTP atraviesa un pipeline de quince middlewares PSR-15 antes de llegar al controlador correspondiente. La figura 2.2 muestra el orden de ejecución.

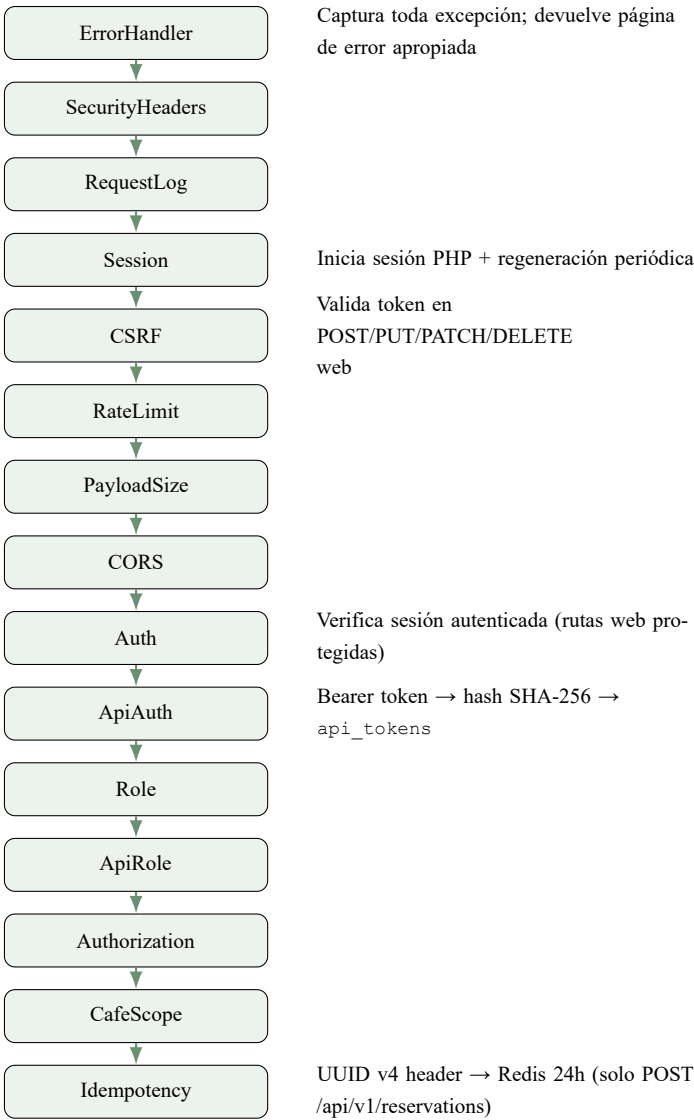


Figura 2.2: Pipeline de 15 middlewares PSR-15

La cuadro 2.1 describe brevemente cada middleware.

Tabla 2.1: Catálogo de middlewares PSR-15

Middleware	Responsabilidad
ErrorHandlerMiddleware	Envuelve toda la cadena; captura Throwable y devuelve respuesta de error apropiada (HTML 4xx/5xx o JSON según el tipo de ruta).

Middleware	Responsabilidad
SecurityHeadersMiddleware	Añade cabeceras de seguridad: X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, Referrer-Policy.
RequestLogMiddleware	Registra método, URI, IP, agente y tiempo de respuesta en stdout (12-Factor XI).
SessionMiddleware	Inicia la sesión PHP, gestiona regeneración periódica del identificador y establece atributos de sesión en el objeto Request.
CsrfMiddleware	Verifica el token CSRF en todas las rutas mutantes (POST/PUT/PATCH/DELETE) web. Rechaza con 419 si el token falta o no coincide.
RateLimitMiddleware	Limitación de velocidad por clave (bucket Redis): login, registration, password_reset. Responde 429 al exceder.
PayloadSizeMiddleware	Rechaza cuerpos de petición superiores a 10 MB para prevenir ataques de agotamiento de memoria.
CorsMiddleware	Gestiona cabeceras CORS y respuestas OPTIONS preflight para la API.
AuthMiddleware	Requiere sesión autenticada. Redirige a /login si no está logado.
ApiAuthMiddleware	Extrae el Bearer token, aplica SHA-256, consulta api_tokens y autentica la petición de forma stateless.
RoleMiddleware	Verifica que el rol del usuario en sesión coincide con el rol requerido por la ruta web.
ApiRoleMiddleware	Igual que RoleMiddleware pero para peticiones API autenticadas con Bearer token.

Middleware	Responsabilidad
AuthorizationMiddleware	Aplica políticas de autorización de grano fino (p.ej. solo el propietario puede modificar sus datos).
CafeScopeMiddleware	Restringe el acceso de managers y supervisors a los datos del café al que están asignados.
IdempotencyMiddleware	Verifica el header <code>Idempotency-Key</code> (UUID v4); devuelve la respuesta cacheada (Redis, TTL 24 h) si existe, o ejecuta y almacena el resultado.

2.3. Flujos de autenticación

El sistema implementa tres mecanismos de autenticación diferenciados según el tipo de cliente.

2.3.1. Login web con sesión

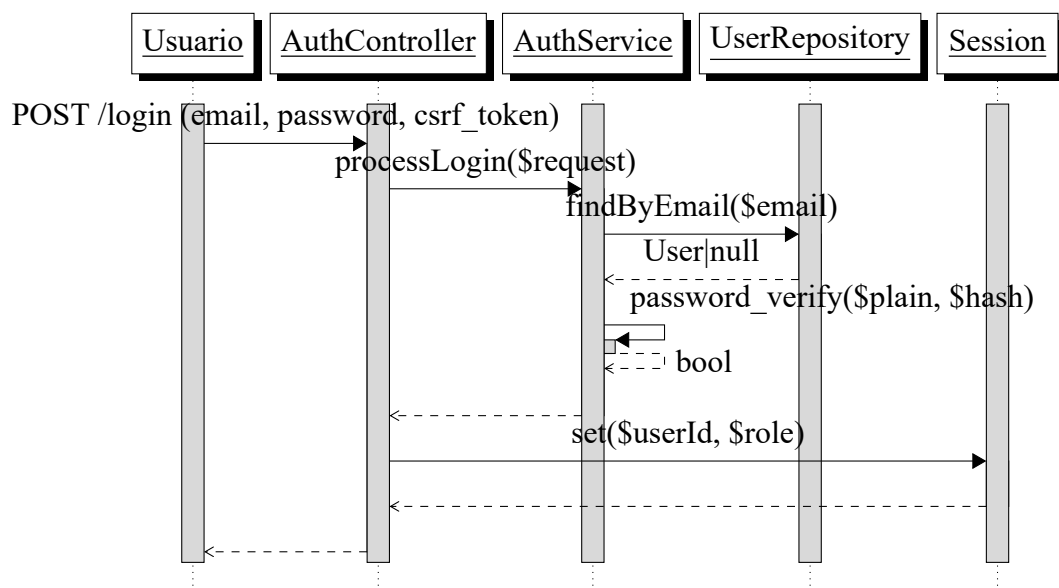


Figura 2.3: Diagrama de secuencia: login web

2.3.2. Verificación Role-Based Access Control (RBAC) por middleware

El `RoleMiddleware` comprueba el rol almacenado en sesión antes de que la petición llegue al controlador. Los roles disponibles son: `admin`, `manager`, `supervisor`, `reception`,

kitchen, keeper y user.

2.3.3. API stateless con Bearer token

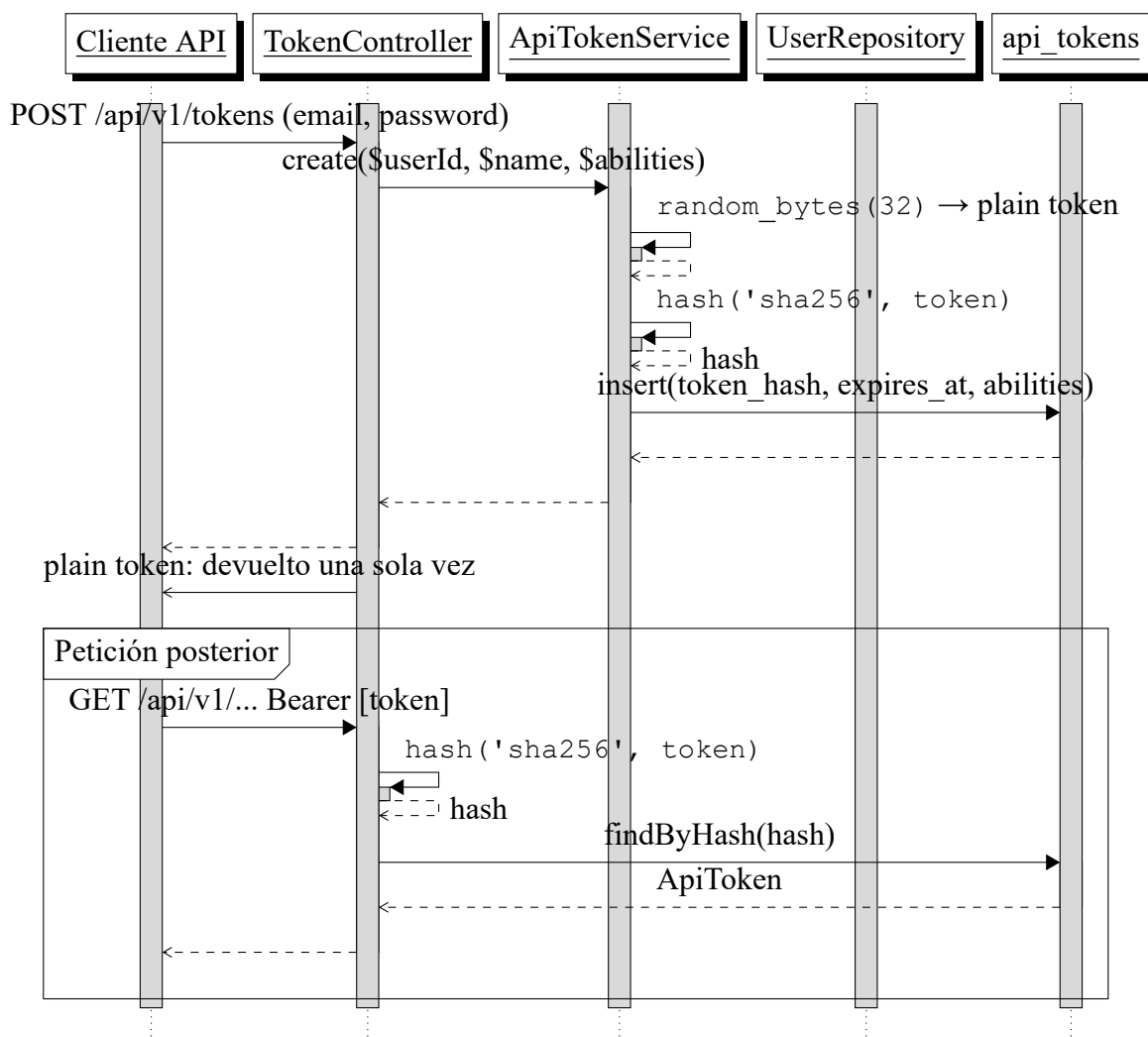


Figura 2.4: Diagrama de secuencia: autenticación Bearer token API

2.4. Flujo de reservas con control de idempotencia

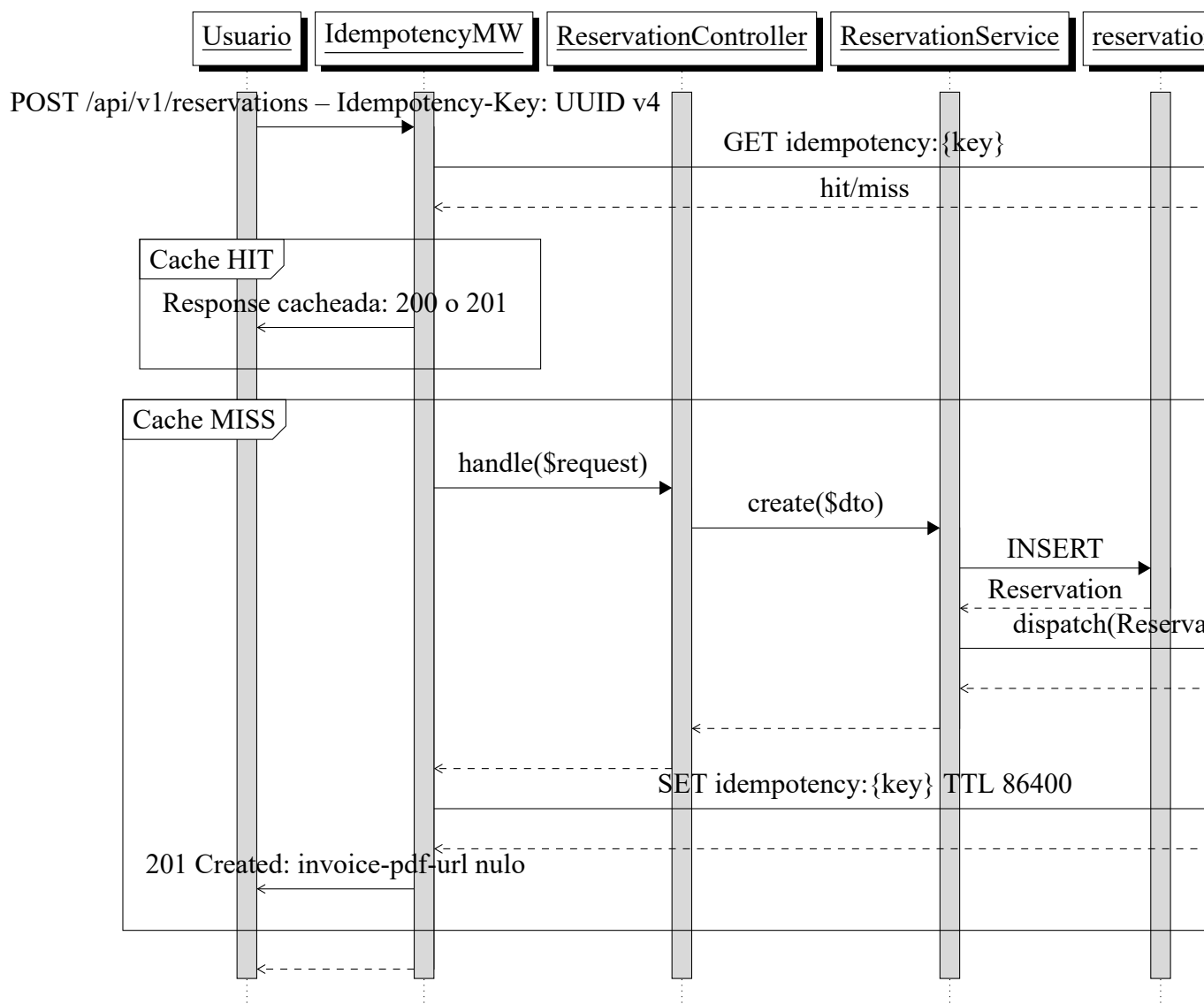


Figura 2.5: Diagrama de secuencia: creación de reserva con idempotencia

Nota

El campo `invoice_pdf_url` es NULL en la respuesta inicial. El `InvoicePDFService` genera el PDF de forma asíncrona (worker Redis) y actualiza la columna `reservations.invoice_pdf_url` cuando el archivo está disponible.

2.5. Modelo entidad-relación

La base de datos está compuesta por 19 migraciones SQL que definen las tablas principales del sistema. La cuadro 2.2 resume las entidades más relevantes.

Tabla 2.2: Entidades principales del modelo de datos

Tabla	Propósito	Columnas clave
users	Cuentas de usuario	id, email, password_hash, role, is_active
cafes	Establecimientos temáticos	id, slug, name, capacity, is_active
products	Carta del menú	id, cafe_id, name, price, is_available
allergens	Alérgenos por producto	id, product_id, name, severity
reservations	Reservas de clientes	id, cafe_id, user_id, date, guests, status, idempotency_key, invoice_pdf_url
api_tokens	Tokens de acceso API	id, user_id, token_hash, abilities, expires_at, revoked_at
loyalty_cards	Tarjetas de fidelización	id, user_id, points, tier
animals	Animales de los cafés	id, cafe_id, name, species, is_active
animal_incidents	Incidencias de animales	id, animal_id, severity, description, resolution
health_checks	Revisiones de salud	id, animal_id, keeper_id, checked_at, notes
reviews	Reseñas de clientes	id, cafe_id, user_id, rating, body, status
newsletter_subscriptions	Suscriptores de newsletter	id, email, verified_at, expires_at

Tabla	Propósito	Columnas clave
staff_shifts	Turnos de personal	id, user_id, cafe_id, start_time, end_time
time_slots	Franjas horarias de reserva	id, cafe_id, date, time, capacity, booked
waitlist	Lista de espera	id, cafe_id, user_id, token, status

El diagrama entidad-relación completo (en formato PlantUML) se encuentra en `docs/diagrams/er.pu` e incluye todas las relaciones de clave foránea y constraints de unicidad.

2.6. Implementación frontend

Las vistas se renderizan en el servidor mediante el motor de plantillas PHP interno (`app/Core/View.php`). Alpine.js proporciona la interactividad del lado del cliente sin necesidad de un *bundler* en producción.

2.6.1. Convenciones de vistas

- Todas las vistas viven en `resources/views/`.
- `View::render()` auto-escapa todos los valores de `$data`.
- Para HTML o JSON pre-procesados se usa `Raw::html()` / `Raw::json()`.
- Los layouts se encuentran en `resources/views/layouts/:main.php` (público/usuario), `backoffice.php` (admin/manager), `kds.php` (cocina), `mobile.php`, `reception.php`.

2.6.2. Patrón de datos para Alpine.js

Listing 2.1: Pasar JSON seguro a Alpine.js

```

1 <?php declare(strict_types=1);
2
3 // En el controlador:
4 $data = [
5     'reservations' => Raw::json($reservationsArray),
6     'cafes'         => Raw::json($cafesArray),
7 ];
8 View::render('admin/reservations/index', $data, ['admin-reservations.css'
9     ]);

```

2.7. Implementación backend

2.7.1. El Result pattern

Todos los servicios devuelven un objeto `Result` en lugar de lanzar excepciones para fallos esperados. Esto hace que el flujo de control sea explícito y predecible.

Listing 2.2: Ejemplo de uso del Result pattern

```
1 <?php declare(strict_types=1);
2
3 namespace App\Services;
4
5 use App\Core\Result;
6
7 final class ReservationService
8 {
9     public function create(ReservationDTO $dto): Result
10     {
11         if ($dto->guests > $this->getCapacity($dto->cafeId)) {
12             return Result::fail('Capacidad insuficiente', '
13                 capacity_exceeded');
14         }
15
16         $id = $this->repository->insert($dto->toArray());
17         return Result::ok(['id' => $id]);
18     }
19 }
```

Listing 2.3: Consumo del Result en el controlador

```
1 <?php declare(strict_types=1);
2
3 $result = $this->service->create($dto);
4
5 if (!$result->ok) {
6     Flash::error($result->getMessage());
7     return $this->response->redirect('/reservar');
8 }
9
10 return $this->response->redirect('/reservas/confirmacion/' . $result->data
    ['id']);
```

2.7.2. Catálogo de rutas

Tabla 2.3: Catálogo de rutas del sistema (selección)

Método	Ruta	Controlador	Middleware
GET	/	Public\HomeController@index	—
GET	/cafes	Public\CafeController@index	—
GET	/cafes/{slug}	Public\CafeController@show	—
GET	/menu	Public\MenuController@index	—
GET	/quiz	Public\QuizController@index	—
POST	/quiz/resultado	Public\QuizController@resultado	csrf
GET	/historia	Public\PageController@historia	—
GET	/faq	Public\PageController@faq	—
GET	/contacto	Public\PageController@contacto	—
GET	/login	Auth\AuthController@showLogin	guest
POST	/login	Auth\AuthController@processLogin	csrf, rateLimit(login)
GET	/registro	Auth\AuthController@showRegister	guest
POST	/registro	Auth\AuthController@processRegister	csrf, rateLimit(registration)
GET	/forgot-password	Auth>PasswordResetController	guest
POST	/forgot-password	Auth>PasswordResetController	csrf, rateLimit(password_reset)
POST	/logout	Auth\AuthController@logout	csrf
GET	/perfil	Shared\UserController@profile	auth
GET	/reservas/mis-reservas	Shared\ReservationController	auth
GET	/loyalty/card	Public\LoyaltyController@card	auth
GET	/mis-favoritos	User\FavoriteController@index	auth
GET	/admin/dashboard	Admin\DashboardController@index	auth, role(admin)

Método	Ruta	Controlador	Middleware
GET	/admin/users	Admin\UserController@index	auth, role(admin)
GET	/admin/cafes	Admin\CafeController@index	auth, role(admin)
GET	/admin/menu	Admin\MenuController@index	auth, role(admin)
GET	/admin/reservations	Admin\ReservationController@index	auth, role(admin)
GET	/admin/animals	Admin\AnimalController@index	auth, role(admin)
GET	/admin/reports	Admin\ReportController@index	auth, role(admin)
GET	/admin/settings	Admin\SystemController@settings	auth, role(admin)
GET	/admin/logs/audit	Admin\AuditLogController@index	auth, role(admin)
GET	/admin/loyalty	Admin\LoyaltyController@index	auth, role(admin)
GET	/manager/dashboard	Manager\DashboardController@index	auth, role(manager)
GET	/manager/products	Manager\ProductController@index	auth, role(manager), ownsCafe
GET	/manager/staff	Manager\StaffController@index	auth, role(manager), ownsCafe
GET	/reception	Reception\ReceptionController@index	auth, role(reception)
GET	/kitchen	Kitchen\KitchenController@index	auth, role(kitchen)
GET	/keeper/dashboard	Keeper\AnimalDashboardController	auth, role(keeper)
GET	/keeper/health-checks	Keeper\HealthCheckController@index	auth, role(keeper)
GET	/api/v1/menu/alergenos	Api\V1\MenuController@allergens	cors
GET	/api/v1/cafes	Api\V1\CafeController@index	cors
GET	/api/v1/time-slots/available	Api\V1\TimeSlotController	cors
POST	/api/v1/reservations	Api\V1\ReservationController@create	cors, apiAuth, csrf, idempotency
GET	/api/v1/tokens	Api\V1\TokenController@list	cors, apiAuth
POST	/api/v1/tokens	Api\V1\TokenController@create	cors, csrf
DELETE	/api/v1/tokens/{id}	Api\V1\TokenController@revoke	cors, apiAuth, csrf

Método	Ruta	Controlador	Middleware
POST	/api/v1/admin/users	Api\V1\Admin\UserApiController@create	apiAuth, role(admin), csrf
PUT	/api/v1/admin/cafes/{id}	Api\V1\Admin\CafeApiController@update	apiAuth, role(admin), csrf
POST	/api/v1/admin/menu	Api\V1\Admin\MenuApiController@create	apiAuth, role(admin), csrf
POST	/api/v1/ops/reservations/{id}/checkin	Api\V1\Ops\ReceptionApiController	apiAuth, role(reception), csrf
POST	/api/v1/ops/orders/{id}/complete	Api\V1\Ops\KitchenApiController	apiAuth, role(kitchen), csrf

2.7.3. Catálogo de servicios de negocio

Tabla 2.4: Catálogo de servicios de negocio (53 servicios)

Servicio	Responsabilidad principal
AuthService	Autenticación web: login, logout, verificación de credenciales
ApiTokenService	Gestión de tokens Bearer: creación, revocación, validación
AuthTokenService	Tokens de verificación de email y reset de contraseña
PasswordResetService	Flujo completo de recuperación de contraseña
EmailVerificationService	Envío y verificación de emails de confirmación de cuenta
SessionManagementService	Listado, revocación y expiración de sesiones activas
UserAccountService	Alta, baja y gestión general de cuentas de usuario
UserManagementService	Operaciones admin sobre usuarios (toggle activo, roles)
UserProfileService	Actualización de perfil, avatar y preferencias de usuario
UserPreferenceService	Preferencias de cookies, filtros dietéticos y recientes
CafeService	CRUD de cafés temáticos y gestión de estado
MenuService	Catálogo de menú: categorías y disponibilidad
ProductService	CRUD de productos de carta con toggle de disponibilidad
AllergenService	Asociación de alérgenos a productos
ReservationService	Creación, confirmación y cancelación de reservas
ReservationTimeSlotService	Validación y gestión de franjas horarias de reserva
AvailabilityService	Cálculo de plazas disponibles por café, fecha y hora
TimeSlotService	CRUD de time slots y estadísticas de ocupación
WaitlistService	Alta en lista de espera, posición y confirmación

Servicio	Responsabilidad principal
LoyaltyService	Acumulación de puntos, canjes y gestión de catálogo
GamificationService	Logros, niveles y recompensas del programa de fidelización
ReviewService	Creación y gestión de reseñas de clientes
ReviewModerationService	Aprobación y rechazo de reseñas por moderadores
ReviewQueryService	Consultas y filtros avanzados sobre reseñas
CartService	Gestión del carrito (Redis, por sesión o usuario)
RecentlyViewedService	Historial de productos visitados recientemente (cookies)
NewsletterService	Suscripción, verificación y baja del boletín
EmailService	Envío de emails transaccionales (queue + SMTP)
InvoicePDFService	Generación asíncrona de facturas PDF (Dompdf)
KitchenService	Órdenes activas e historial del KDS de cocina
ReceptionService	Check-in, checkout y procesamiento de pagos en recepción
AnimalCareService	CRUD de animales, incidentes y revisiones de salud
HealthCheckService	Registro y consulta de revisiones veterinarias
StaffShiftService	Gestión de turnos de personal por café
SupervisorAssignmentService	Asignaciones de supervisor a café y turnos
SettingsService	Lectura y escritura de configuración del sistema
CacheService	Abstracción de Redis con tags e invalidación por grupo
RateLimitingService	Implementación de ventana deslizante en Redis
FileUploadService	Subida y validación de imágenes (avatares, animales)
CloudinaryStorageService	Almacenamiento de imágenes en Cloudinary (producción)
TelegramService	Notificaciones push al bot de Telegram del equipo
MercurePublisherService	Publicación de eventos SSE en el hub Mercure (KDS)
WeatherService	Consulta de meteorología local (API externa)

Servicio	Responsabilidad principal
NavigationService	Generación del menú de navegación según rol activo
AdminActivityService	Registro de acciones de administración en audit log
AdminStatisticsService	Cálculo de KPIs para el dashboard de administración
AdminReportService	Generación y exportación de informes en CSV/PDF
AccountDeletionService	Eliminación segura de cuentas de usuario y datos asociados
ClimaContextoService	Contexto climático (temperatura, lluvia) para personalizar la experiencia
ContextServiceInstance	Versión inyectable de ContextService: resuelve el café activo por sesión
DashboardService	Métricas en tiempo real para el panel de control del gestor
FestivosJaponesesService	Consulta de festivos japoneses (祝日 Shukujitsu) para validación de slots
MicroestacionesService	Calendario solar japonés (24 Sekki) para personalización estacional

2.7.4. Catálogo de repositorios

Tabla 2.5: Catálogo de repositorios (29 repositorios)

Repositorio	Entidad / Tabla gestionada
UserRepository	users
RoleRepository	roles, user_roles
SessionRepository	sessions
AuthTokenRepository	auth_tokens (email/reset)
ApiTokenRepository	api_tokens (Bearer)
CafeRepository	cafes
MenuCategoryRepository	menu_categories
MenuRepository	menu_items
ProductRepository	products
AllergenRepository	allergens

Repositorio	Entidad / Tabla gestionada
ReservationRepository	reservations
ReservationItemRepository	reservation_items
TimeSlotRepository	time_slots
WaitlistRepository	waitlist
ReviewRepository	reviews
LoyaltyRepository	loyalty_cards, loyalty_transactions
FavoriteRepository	favorites
NewsletterSubscriptionRepository	newsletter_subscriptions
AnimalRepository	animals
AnimalIncidentRepository	animal_incidents
HealthCheckRepository	health_checks
StaffShiftRepository	staff_shifts
SupervisorAssignmentRepository	supervisor_assignments
SettingRepository	system_settings
AuditLogRepository	audit_logs
AuthLogRepository	auth_logs
StatisticsRepository	Vistas agregadas / consultas analytics
TrackerRepository	interaction_sessions
InteractionSessionRepository	Sesiones de interacción (A/B testing)

2.8. Pruebas

2.8.1. Tests unitarios con PHPUnit

Los tests unitarios se ubican en `tests/Unit/` replicando la estructura de `app/`. Cada archivo de test incluye el docblock obligatorio del proyecto:

Listing 2.4: Estructura de test unitario

```

1 <?php declare(strict_types=1);
2
3 /**
4  * ¿Qué pruebas aquí?
5  *
6  * El ReservationService crea reservas correctamente y falla
7  * con gracia cuando se supera la capacidad.
8  * ¿Qué me quieres demostrar?
9  *
10 * Que el Result pattern encapsula los errores sin excepciones.

```

```
9  * ¿Qué va a fallar en este test si se cambia el código?
10 *   Si create() lanza una excepción en lugar de devolver Result::fail().
11 */
12 class ReservationServiceTest extends TestCase
13 {
14     public function testCreateReturnsOkWhenCapacityIsAvailable(): void
15     {
16         $repo = $this->createStub(ReservationRepository::class);
17         $repo->method('insert')->willReturn(42);
18
19         $service = new ReservationService($repo, /* ... */);
20         $result = $service->create($dto);
21
22         $this->assertTrue($result->ok);
23         $this->assertSame(42, $result->data['id']);
24     }
25 }
```

2.8.2. Tests de integración

Los tests de integración en `tests/Integration/` utilizan una base de datos real (`docker-compose.t` con migraciones y seeders ejecutados antes de cada suite. Se ejecutan con:

Listing 2.5: Ejecutar tests de integración

```
1 make test-integration
```

2.8.3. Tests E2E con Playwright

Los tests end-to-end se encuentran en `tests/e2e/` y están escritos en TypeScript. Cubren flujos completos: login, creación de reserva, check-in y verificación de KDS. Se ejecutan con:

Listing 2.6: Ejecutar tests E2E

```
1 make e2e
2 make e2e-all
```

2.9. Estructura de directorios

```
komorebi/
├── app/
│   ├── Core/
│   │   └── Router, Container, View, DB, Cache, Queue, Logger
│   ├── Http/
│   │   └── Controllers/
│   │       └── Admin/ Manager/ Kitchen/ Keeper/ ...
│   ├── Services/
│   │   └── 53 servicios de negocio
│   ├── Repositories/
│   │   └── 29 repositorios PDO
│   ├── Events/ + Listeners/
│   └── routes.php
├── bootstrap/
│   └── container.php
├── migrations/
│   └── 001 -- 019 SQL
├── resources/
│   └── views/
│       └── layouts/ admin/ public/ ...
├── tests/
│   └── Unit/ Integration/ e2e/
├── public/
│   └── index.php css/ js/ images/
└── docs/
    └── diagrams/ images/ screenshots/ plans/
```

Figura 2.6: Estructura de directorios del proyecto

2.10. Despliegue

2.10.1. Entorno local con Docker Compose

Listing 2.7: Arrancar el entorno de desarrollo

```
1 make dev
2 make db-migrate
3 make db-seed
4 make test
```

El archivo `docker-compose.yml` define cuatro servicios:

- `app` —FrankenPHP (Caddy + PHP 8.4 integrado), puerto 8080
- `mysql` —MySQL 8.0, puerto 3306
- `redis` —Redis 7, puerto 6379
- `mailpit` —SMTP local + UI, puertos 1025/8025

2.10.2. Producción en Railway

El despliegue a producción se realiza automáticamente mediante GitHub Actions al hacer *push* a `main`. Railway detecta el `Dockerfile` (multi-stage build) y despliega la imagen resultante.

Las variables de entorno sensibles se gestionan como secretos en Railway. El helper `SecretLoader::require()` busca la variable de entorno primero y después el archivo `/run/secrets/<clave>` (estándar Docker Swarm).

2.11. Control de versiones y Git Flow

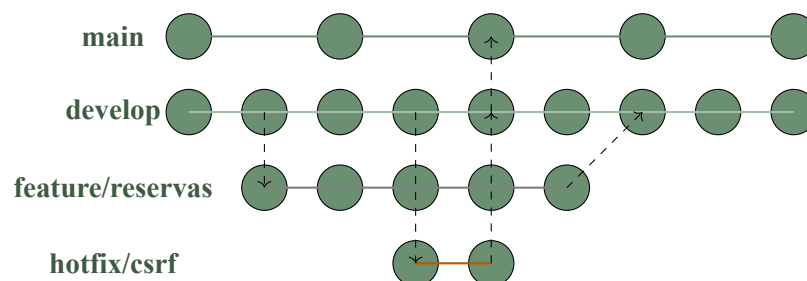


Figura 2.7: Flujo de ramas Git Flow del proyecto

Capítulo 3

Manual de Usuario

Este capítulo describe las funcionalidades de la plataforma desde la perspectiva del usuario final, organizadas por tipo de acceso y rol. Cada sección incluye capturas de pantalla reales del sistema.

Nota

Las capturas de pantalla están numeradas del 01 al 62 y se encuentran en docs/screenshots/. Se referencian mediante `\cref{fig:NN-nombre}` a lo largo del capítulo.

3.1. Área pública

El área pública es accesible sin autenticación y muestra la presentación de la cadena, los cafés disponibles, la carta del menú, el sistema de reservas, la historia de la marca y los recursos informativos.

3.1.1. Página de inicio

La página principal (figura 3.1) presenta el concepto de la marca mediante un encabezado animado con el significado de *komorebi*, seguido de las secciones destacadas: cafés disponibles, menú del momento y acceso rápido a reservas.

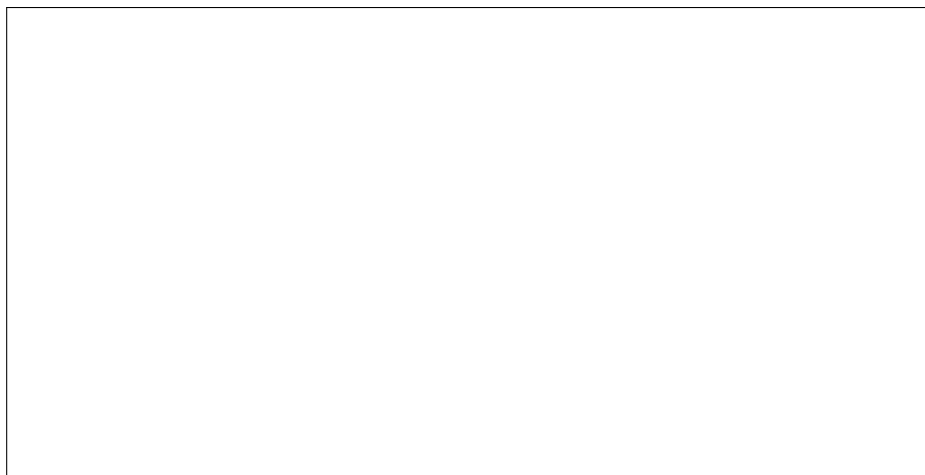


Figura 3.1: Página de inicio —Komorebi Café

3.1.2. Catálogo de cafés

La página `/cafes` (figura 3.2) muestra las tarjetas de los catorce cafés temáticos con imagen, nombre y botón de acceso al detalle.

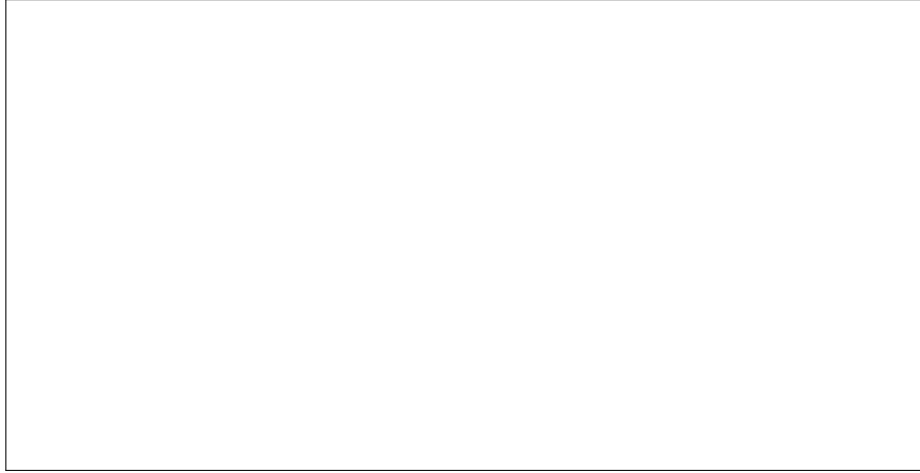


Figura 3.2: Catálogo de cafés temáticos

3.1.3. Detalle de café

Cada café dispone de su propia página de detalle (figura 3.3) con galería de imágenes, descripción temática, horarios, capacidad y acceso directo al formulario de reserva.

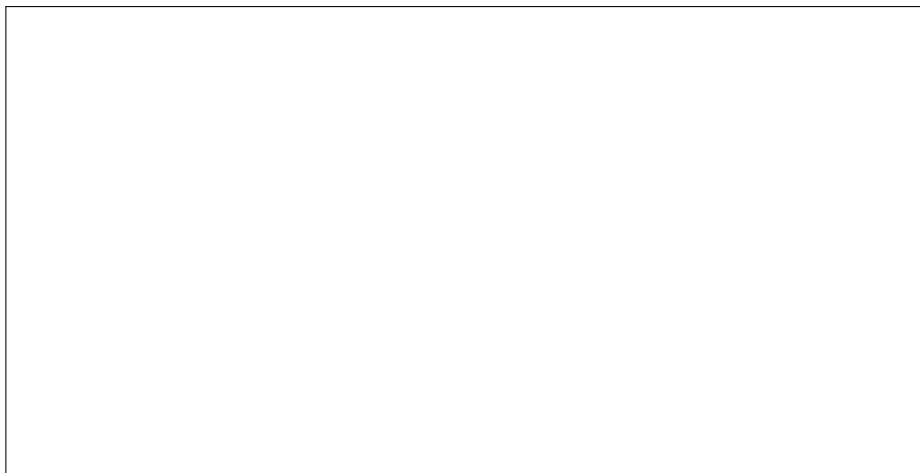


Figura 3.3: Página de detalle de un café temático

3.1.4. Carta del menú

La carta digital (figura 3.4) presenta todos los productos con foto, descripción y precio. Incluye filtros por categoría y por alérgeno en tiempo real mediante Alpine.js.



Figura 3.4: Carta digital del menú con filtros por alérgeno

3.1.5. Quiz de recomendación

El quiz (figura 3.5) guía al visitante a través de una serie de preguntas sobre preferencias estéticas y recomienda el café temático más afín a su personalidad.

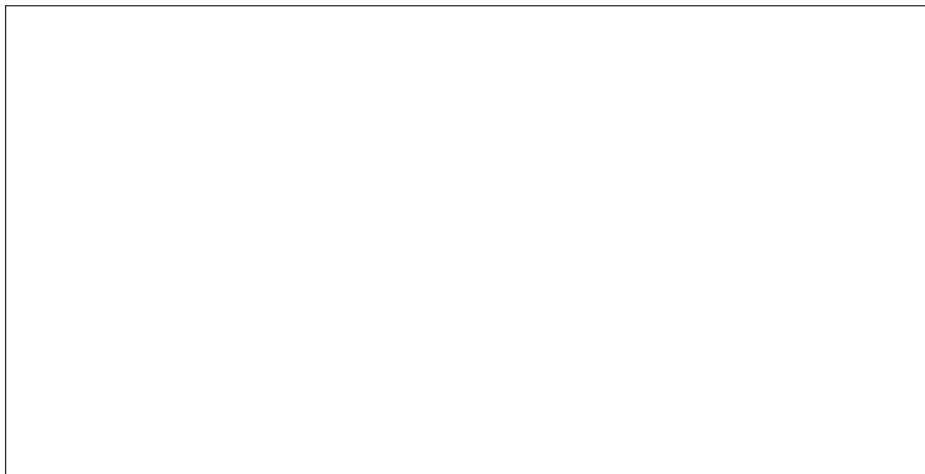


Figura 3.5: Quiz de recomendación de café

3.1.6. Historia de la marca

La página `/historia` (figura 3.6) narra el origen del concepto *komorebi* y la filosofía que inspira cada uno de los establecimientos.



Figura 3.6: Página de historia de la marca

3.2. Autenticación y gestión de cuenta

3.2.1. Formulario de login

El formulario de acceso (figura 3.7) solicita email y contraseña. Incluye protección Cross-Site Request Forgery (CSRF) y límite de velocidad de 5 intentos por minuto. El campo contraseña dispone de botón de alternancia de visibilidad.

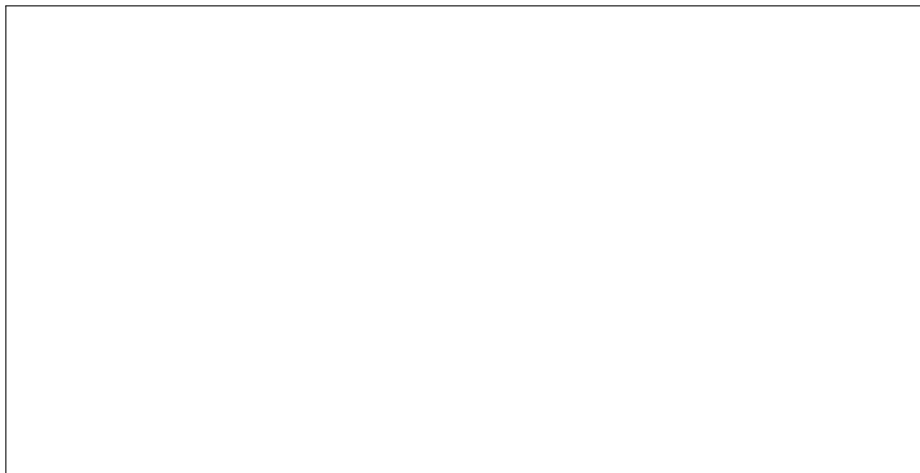


Figura 3.7: Formulario de inicio de sesión

3.2.2. Registro de nueva cuenta

El formulario de registro (figura 3.8) recoge nombre, email y contraseña. Tras el envío, el sistema envía un email de verificación.



Figura 3.8: Formulario de registro de nueva cuenta

3.2.3. Recuperación de contraseña

El flujo de recuperación (figura 3.9) envía un enlace de restablecimiento con validez de 60 minutos al email registrado.

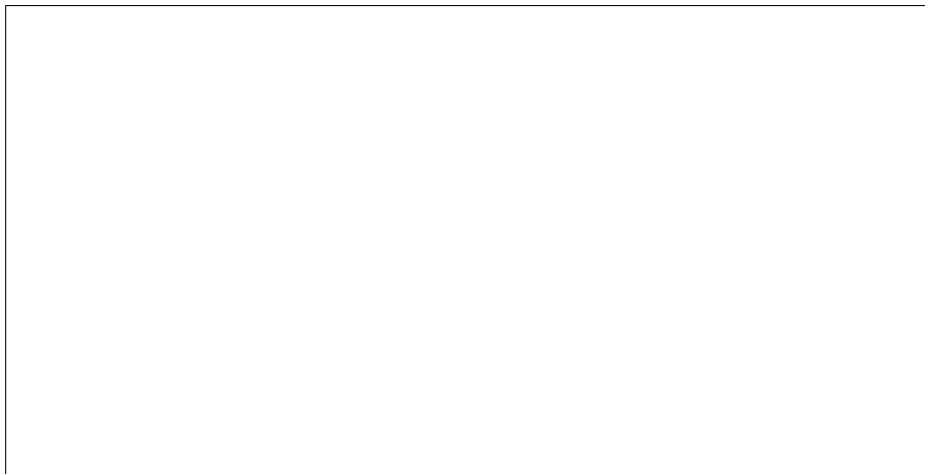


Figura 3.9: Formulario de recuperación de contraseña

3.3. Usuario autenticado

Una vez autenticado, el usuario accede a su panel personal con reservas, programa de fidelización, favoritos y carrito.

3.3.1. Perfil de usuario

La página `/perfil` (figura 3.10) permite actualizar nombre, email, avatar y contraseña. Los cambios de contraseña requieren confirmación de la contraseña actual.



Figura 3.10: Página de perfil de usuario

3.3.2. Mis reservas

La sección de reservas (figura 3.11) lista todas las reservas del usuario con estado (pendiente, confirmada, cancelada), fecha, café y número de comensales. Permite cancelar reservas con más de 24 horas de antelación.

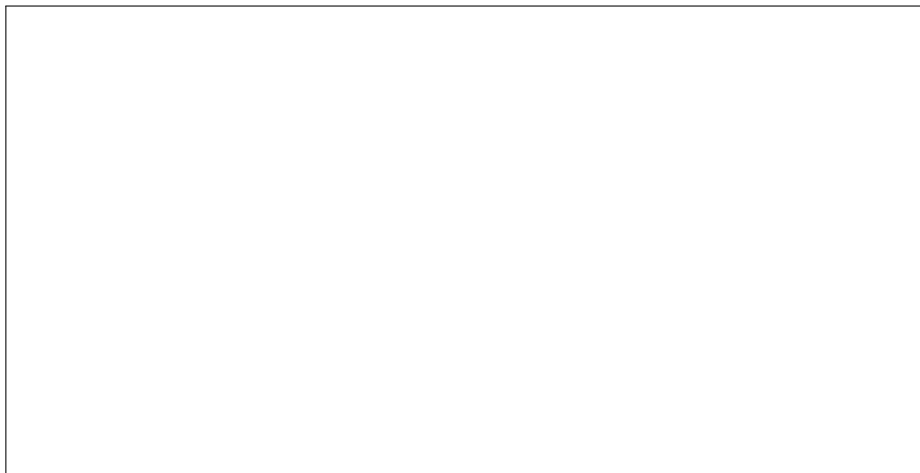


Figura 3.11: Listado de reservas del usuario

3.3.3. Nueva reserva

El asistente de reserva en dos pasos (figura 3.12) muestra primero la selección de café, fecha y franja horaria disponible. En el segundo paso (figura 3.13) se confirma el número de comensales y las preferencias dietéticas.

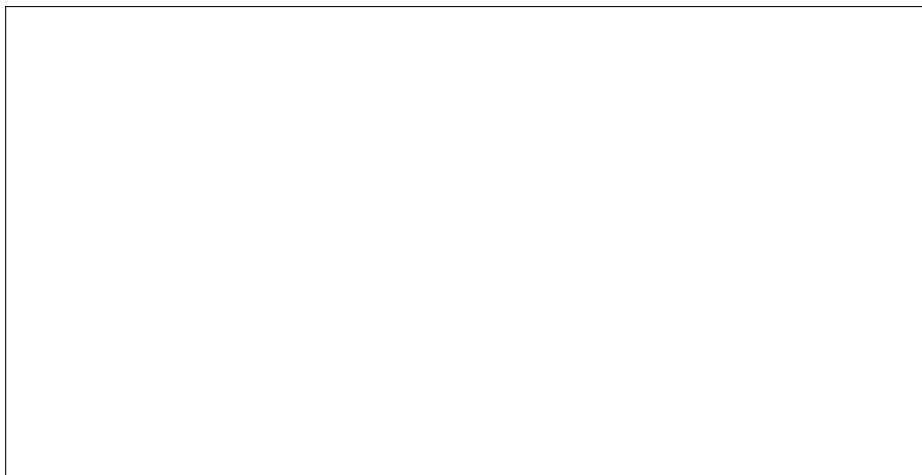


Figura 3.12: Formulario de reserva —paso 1

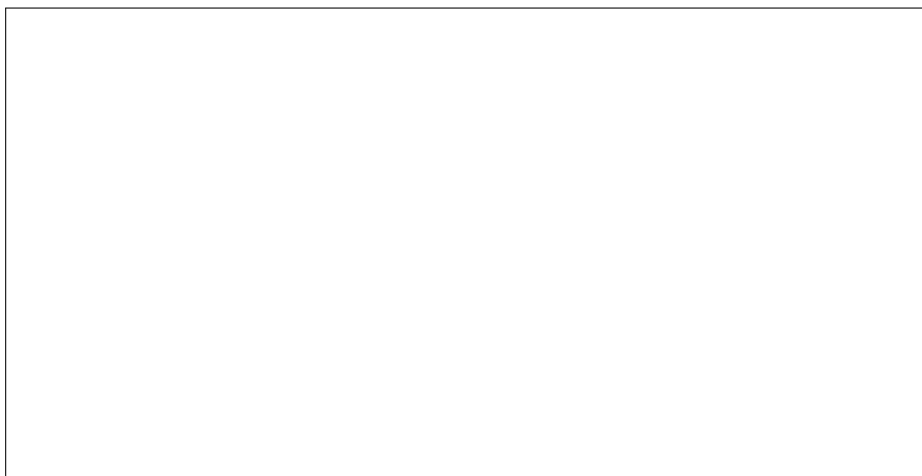


Figura 3.13: Formulario de reserva —paso 2

3.3.4. Tarjeta de fidelización

La tarjeta digital (figura 3.14) muestra los puntos acumulados, el nivel (*bronze*, *silver*, *gold* o *platinum*), el historial de transacciones y los premios canjeables del catálogo.

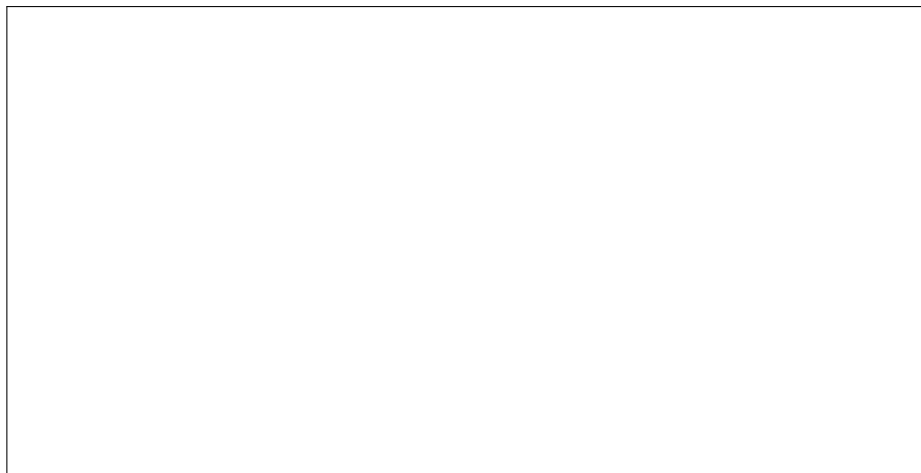


Figura 3.14: Tarjeta de fidelización digital

3.4. Panel de administración

El backoffice de administración (/admin) está reservado al rol `admin` y proporciona control total sobre todas las entidades del sistema.

3.4.1. Dashboard de administración

El dashboard (figura 3.15) muestra KPIs en tiempo real: reservas del día, ingresos semanales, nuevos usuarios, alertas de animales y estado del sistema.

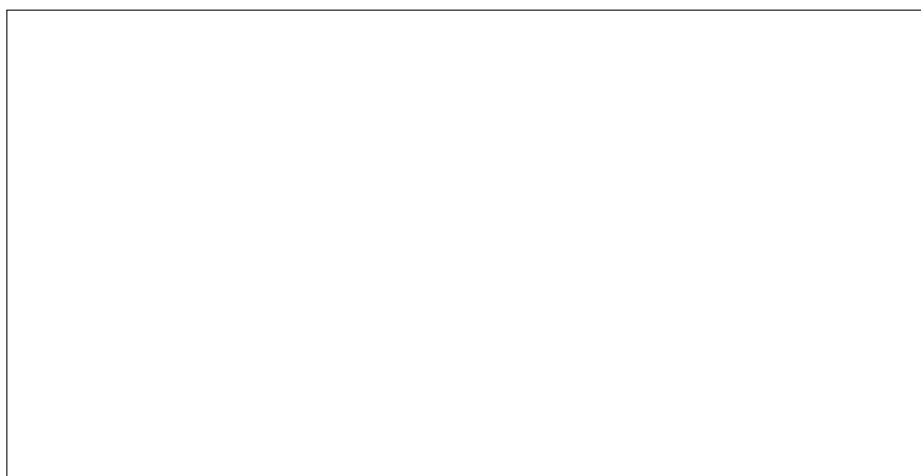


Figura 3.15: Dashboard de administración

3.4.2. Gestión de usuarios

La tabla de usuarios (figura 3.16) permite buscar, filtrar por rol, activar/desactivar cuentas y acceder al detalle de cada usuario.



Figura 3.16: Panel de gestión de usuarios

3.4.3. Gestión de cafés

La administración de cafés (figura 3.17) permite crear nuevos establecimientos, editar sus datos (capacidad, horarios, temática) y activar o desactivar su presencia pública.

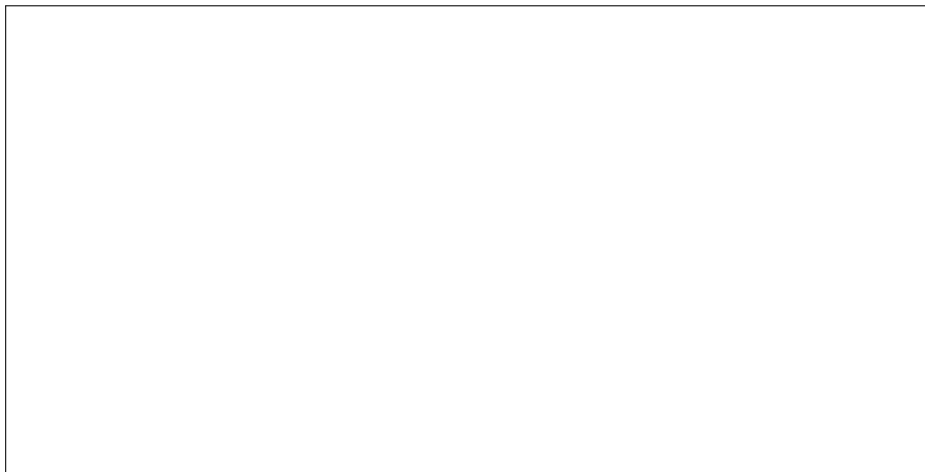


Figura 3.17: Panel de gestión de cafés

3.4.4. Gestión del menú

La carta digital (figura 3.18) permite administrar categorías y productos, definir precios, alérgenos y disponibilidad por café.



Figura 3.18: Panel de gestión del menú

3.4.5. Gestión de reservas

El panel de reservas (figura 3.19) ofrece una vista tabular de todas las reservas con filtros por café, fecha, estado y usuario. Permite confirmar o cancelar reservas individualmente.

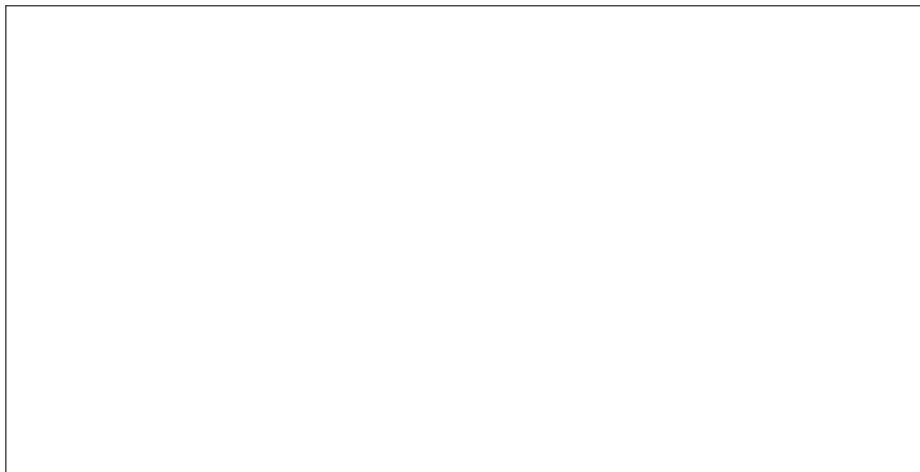


Figura 3.19: Panel de gestión de reservas

3.4.6. Gestión de animales

La sección de animales (figura 3.20) lista todos los residentes de los cafés con su especie, estado de salud, café de origen y acceso al historial de incidentes y revisiones.

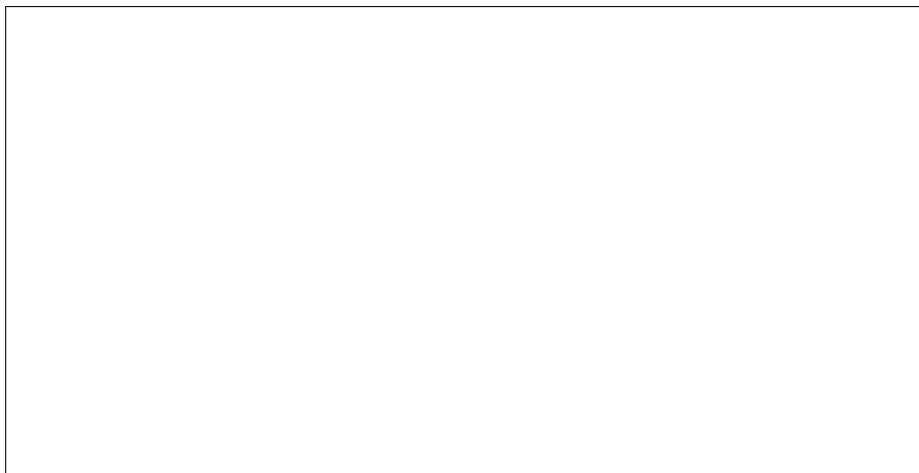


Figura 3.20: Panel de gestión de animales

3.4.7. Logs de auditoría

El visor de logs (figura 3.21) registra todas las acciones administrativas con usuario, timestamp, tipo de acción y entidad afectada. Permite exportar en CSV.

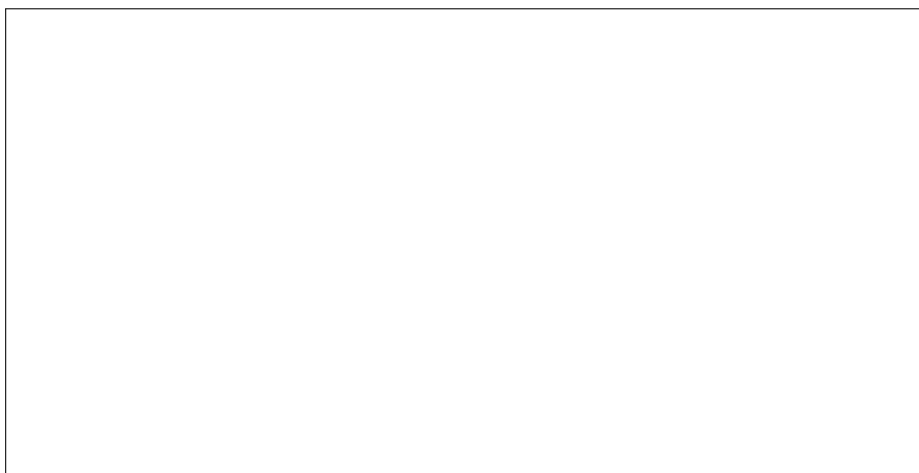


Figura 3.21: Visor de logs de auditoría

3.4.8. Configuración del sistema

La pantalla de configuración (figura 3.22) agrupa los parámetros del sistema por grupos: general, email, reservas, fidelización y mantenimiento.



Figura 3.22: Pantalla de configuración del sistema

3.5. Panel de manager

El manager gestiona las operaciones diarias de su café asignado: productos, staff, reservas e informes.

3.5.1. Dashboard de manager

El dashboard (figura 3.23) muestra el resumen del día: reservas confirmadas, ocupación actual, ingresos y alertas de personal.

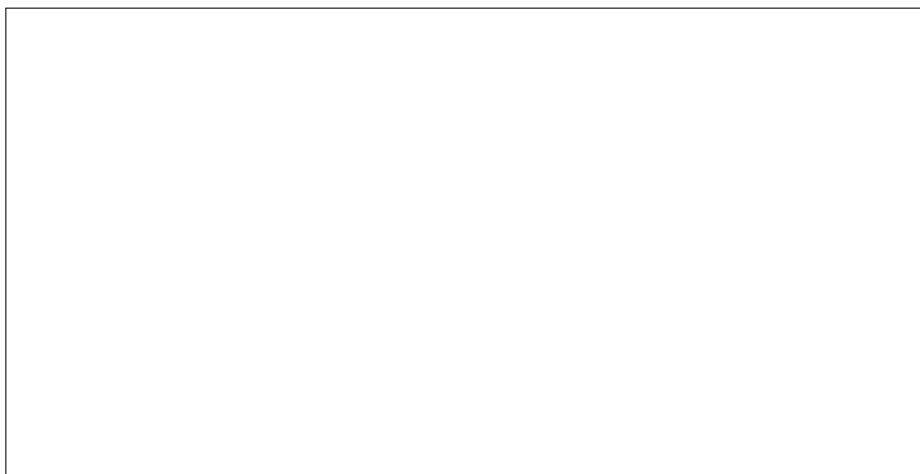


Figura 3.23: Dashboard del manager

3.5.2. Gestión de productos

La carta del manager (figura 3.24) permite activar/desactivar productos específicos de su café sin necesidad de acudir al administrador.



Figura 3.24: Gestión de productos del manager

3.6. Recepción y Cocina

3.6.1. Panel de recepción

El panel de recepción (figura 3.25) muestra en tiempo real las reservas del día con sus estados: esperando, sentada, en consumo, pagada. Permite hacer check-in con un clic.

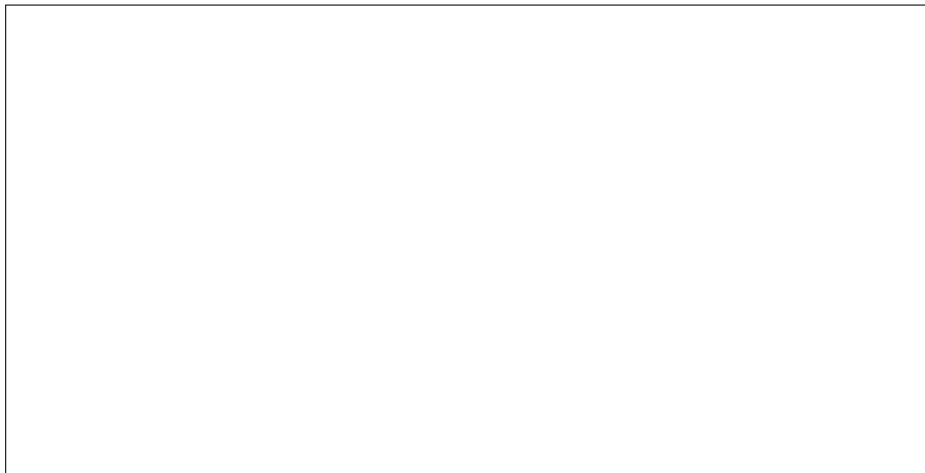


Figura 3.25: Panel de recepción con reservas del día

3.6.2. Kitchen Display System (KDS)

El KDS (figura 3.26) muestra las órdenes activas en tiempo real mediante Server-Sent Events (Mercure). Cada tarjeta incluye el número de mesa, los ítems y el tiempo transcurrido desde la entrada del pedido.



Figura 3.26: Kitchen Display System —órdenes activas

3.7. Panel del Keeper

El rol `keeper` gestiona el bienestar de los animales residentes de los cafés temáticos.

3.7.1. Dashboard del Keeper

El dashboard (figura 3.27) muestra el estado general de todos los animales activos: revisiones pendientes, incidentes abiertos y alertas de salud.

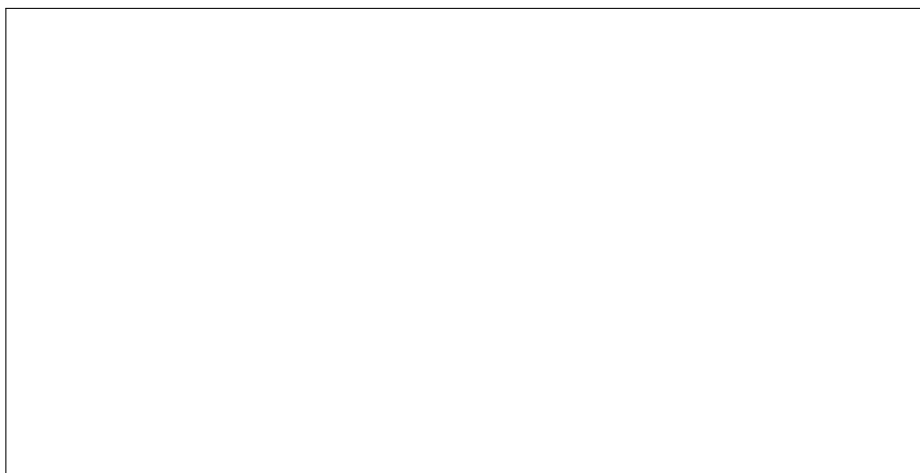


Figura 3.27: Dashboard del Keeper con estado de animales

3.7.2. Ficha de animal

La ficha individual (figura 3.28) incluye datos biográficos, historial de revisiones, incidentes pasados y foto del animal.

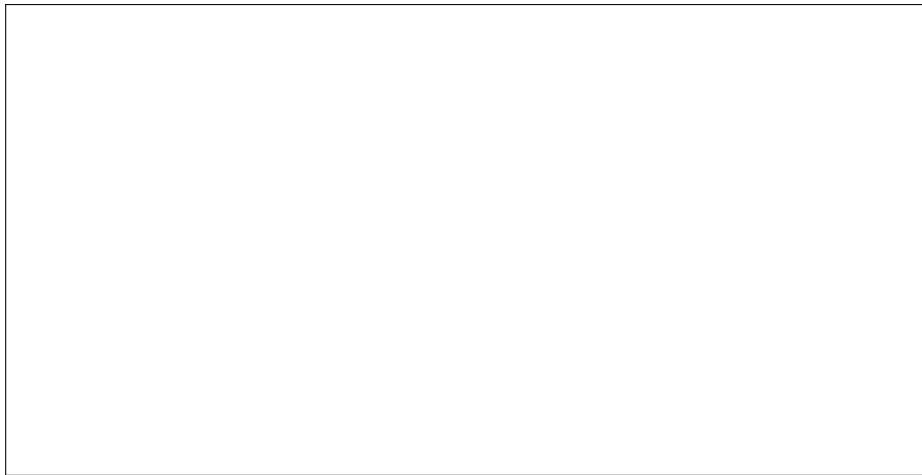


Figura 3.28: Catálogo de animales del café

3.7.3. Revisiones de salud

El formulario de revisión (figura 3.29) registra el estado general, peso, temperatura, observaciones y la fecha de la próxima revisión.

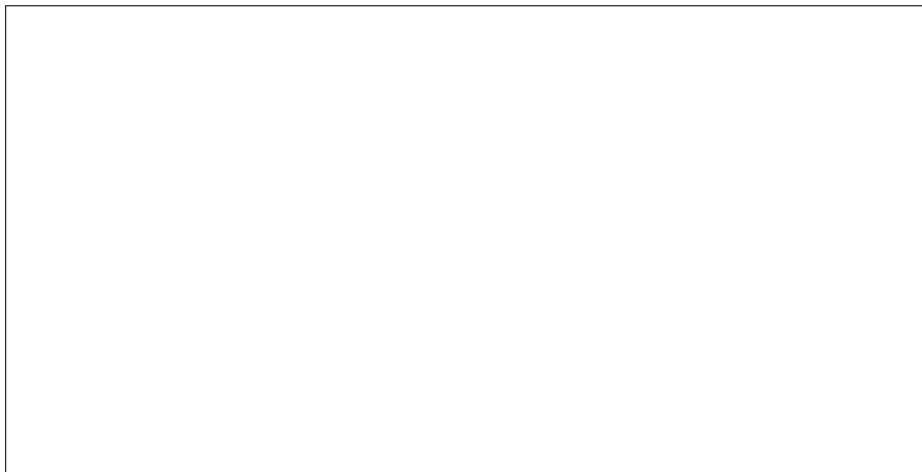


Figura 3.29: Formulario de nueva revisión de salud

3.7.4. Registro de incidentes

El historial de incidentes (figura 3.30) categoriza cada evento por severidad (`low`, `medium`, `high`, `critical`), su descripción, la resolución aplicada y la fecha de cierre.

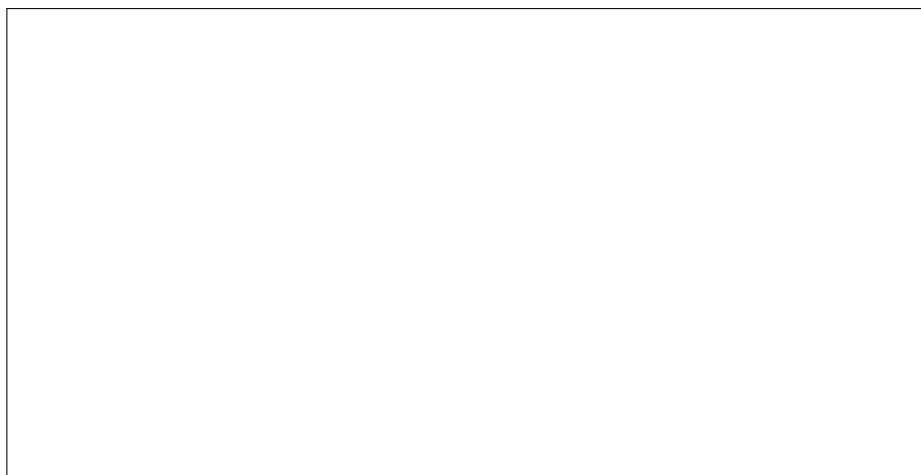


Figura 3.30: Registro de incidentes de animales

Capítulo 4

Conclusiones y Futuras Mejoras

4.1. Objetivos alcanzados

Al inicio del proyecto se plantearon los siguientes objetivos principales, todos ellos cumplidos en la versión actual:

1. **Plataforma funcional de gestión integral:** el sistema cubre el ciclo completo de operaciones de la cadena, desde la reserva pública hasta el cierre de caja en recepción.
2. **Framework MVC propio sobre PHP 8.4:** se ha demostrado la comprensión profunda de los mecanismos subyacentes al construir el router, el contenedor IoC, el motor de vistas y el sistema de middleware desde cero.
3. **Arquitectura orientada a estándares:** adherencia completa a PSR-7, PSR-14 y PSR-15; Result pattern en todos los servicios; DTOs `final readonly`; inyección por interfaz en todos los controladores.
4. **Calidad verificada:** PHPStan nivel 5, Psalm, suite PHPUnit con tests unitarios y de integración, tests E2E con Playwright, análisis de cobertura.
5. **Despliegue en producción:** imagen Docker multi-stage desplegada en Railway con GitHub Actions como pipeline CI/CD.
6. **Identidad visual coherente:** el diseño de la interfaz refleja la estética *komorebi*: paleta natural, tipografía serena y microanimaciones evocadoras.

4.2. Aprendizajes técnicos

El desarrollo de Komorebi Café ha permitido profundizar en áreas que habitualmente quedan ocultas detrás de los frameworks de alto nivel:

PSR-7 y el ciclo petición-respuesta

Trabajar directamente con `ServerRequestInterface` y `ResponseInterface` obliga a entender qué hace realmente un framework cuando procesa una petición HTTP.

El Result pattern como alternativa a las excepciones

Diseñar todos los servicios para que devuelvan `Result::ok()/fail()` en lugar de lanzar excepciones para fallos esperados ha simplificado el flujo de control y hecho el código más predecible.

Redis como infraestructura transversal

Usar Redis no solo para caché sino también para colas de jobs, sesiones, rate-limiting e idempotencia ha demostrado la versatilidad de esta herramienta.

12-Factor en la práctica

Externalizar toda la configuración en variables de entorno, escribir logs a stdout y construir la imagen Docker de forma que sea idéntica en dev y producción simplifica enormemente el ciclo de despliegue.

Accesibilidad como requisito, no como mejora

Integrar los tests de accesibilidad WCAG 2.1 AA en el pipeline desde el principio evita deuda técnica acumulada y garantiza una experiencia inclusiva.

4.3. Futuras mejoras propuestas

1. **Aplicación móvil nativa:** iOS + Android consumiendo la API REST ya disponible, con notificaciones push mediante Firebase Cloud Messaging.
2. **Motor de recomendación:** sistema de recomendación de cafés y productos basado en el historial de reservas y las preferencias del programa de fidelización.
3. **Integración con pasarela de pago:** Stripe o Redsys para cobro online en el momento de la reserva.
4. **Panel de analytics avanzado:** integración con Grafana y Prometheus para monitorización de métricas de negocio en tiempo real.
5. **Internacionalización (i18n):** soporte multiidioma (español, inglés, japonés) para el área pública y la carta digital.
6. **Tests de mutación:** implementar Infection PHP para medir la efectividad real de la suite de tests unitarios.
7. **GraphQL para la API pública:** complementar la API REST con un endpoint GraphQL que permita queries más flexibles desde aplicaciones móviles.
8. **Modo offline en el KDS:** Service Worker para que el KDS de cocina siga operativo durante cortes de conectividad.

Capítulo 5

Referencias Bibliográficas

Apéndice A

Catálogo completo de endpoints API REST

La cuadro A.1 recoge todos los endpoints de la API REST pública y privada de Komorebi Café. Los endpoints marcados con † requieren autenticación Bearer token.

Tabla A.1: Catálogo completo de endpoints API REST (/api/v1/)

Método	Ruta	Auth	Descripción
GET	/menu/alergenos	—	Lista de alérgenos
GET	/menu/productos	—	Catálogo de productos
GET	/menu/products/{id}	—	Detalle de producto
GET	/cafes	—	Lista de cafés activos
GET	/cafes/{slug}	—	Detalle de café
GET	/holidays	—	Festivos del año
GET	/holidays/{date}	—	Verifica si una fecha es festivo
GET	/time-slots/available	—	Franjas disponibles por café y fecha
GET	/time-slots/stats	—	Estadísticas de ocupación
GET	/passes	—	Tipos de pase disponibles
PATCH	/cookies	—	Guardar consentimiento de cookies
GET	/cookies/filters	—	Obtener filtros de dieta
PUT	/cookies/filters	—	Guardar filtros de dieta
DELETE	/cookies/filters	—	Eliminar filtros
PUT	/cookies/dietary	—	Guardar preferencias dietéticas
POST	/cookies/recently-viewed	—	Añadir producto a recientes
GET	/cookies/recently-viewed/data	—	Obtener datos de recientes
DELETE	/cookies/recently-viewed	—	Limpiar recientes

Método	Ruta	Auth	Descripción
POST	/newsletter/subscriptions	—	Suscribirse al boletín
GET	/waitlists/{token}	—	Posición en lista de espera
POST	/waitlists/{token}/confirmations	—	Confirmar posición
POST	/waitlists	†	Unirse a lista de espera
GET	/tokens	†	Listar tokens propios
POST	/tokens	—	Crear token Bearer
DELETE	/tokens/{id}	†	Revocar token
GET	/favorites	†	Listar favoritos
PUT	/favorites/{id}	†	Añadir favorito
DELETE	/favorites/{id}	†	Eliminar favorito
GET	/cart	†	Obtener carrito
GET	/cart/guest	—	Carrito de invitado
POST	/cart/items	†	Añadir ítem
PATCH	/cart/items/{id}	†	Actualizar cantidad
DELETE	/cart/items/{id}	†	Eliminar ítem
DELETE	/cart/items	†	Vaciar carrito
GET	/reservations/available	—	Huecos disponibles
POST	/reservations	†	Crear reserva (idempotente)
GET	/reservations/{id}	†	Detalle de reserva
POST	/reservations/{id}/cancel	†	Cancelar reserva
GET	/user/reservations	†	Mis reservas
GET	/loyalty/validate/{code}	†	Validar código de canje
POST	/loyalty/use	†	Usar puntos

Método	Ruta	Auth	Descripción
POST	/loyalty/redeem	†	Canjear recompensa
POST	/reviews	†	Crear reseña
PUT	/reviews/{id}	†	Actualizar reseña
DELETE	/reviews/{id}	†	Eliminar reseña
GET	/user/profile	†	Perfil del usuario
GET	/user/stats	†	Estadísticas del usuario
GET	/user/reviews	†	Reseñas del usuario
GET	/user/avatar-options	†	Opciones de avatar
GET	/manager/stats	†	KPIs del manager
GET	/manager/revenue/weekly	†	Ingresos semanales
GET	/manager/animals/top	†	Animales más populares
GET	/manager/reservations/status	†	Estado reservas del café
POST	/admin/users	†	Crear usuario
PUT	/admin/users/{id}	†	Actualizar usuario
DELETE	/admin/users/{id}	†	Eliminar usuario
PATCH	/admin/users/{id}/status	†	Toggle activo/inactivo
POST	/admin/cafes	†	Crear café
PUT	/admin/cafes/{id}	†	Actualizar café
DELETE	/admin/cafes/{id}	†	Eliminar café
PATCH	/admin/cafes/{id}/status	†	Toggle estado café
POST	/admin/menu	†	Crear producto
PUT	/admin/menu/{id}	†	Actualizar producto
DELETE	/admin/menu/{id}	†	Eliminar producto

Método	Ruta	Auth	Descripción
PATCH	/admin/menu/{id}/toggle	†	Toggle disponibilidad
POST	/admin/reservations/{id}/confirm	†	Confirmar reserva
POST	/admin/reservations/{id}/cancel	†	Cancelar reserva (admin)
POST	/admin/reviews/{id}/approve	†	Aprobar reseña
POST	/admin/reviews/{id}/reject	†	Rechazar reseña
DELETE	/admin/reviews/{id}	†	Eliminar reseña (admin)
DELETE	/admin/newsletter/subscribers/{email}	†	Dar de baja
GET	/admin/newsletter/export	†	Exportar suscriptores CSV
GET	/admin/loyalty/stats	†	Estadísticas de fidelización
GET	/admin/loyalty/cards	†	Listado de tarjetas
GET	/admin/loyalty/catalog	†	Catálogo de recompensas
PATCH	/admin/loyalty/catalog/{id}/toggle	†	Toggle recompensa
GET	/admin/loyalty/redemptions	†	Historial de canjes
GET	/admin/logs/audit	†	Logs de auditoría
GET	/admin/logs/audit/export	†	Exportar audit log CSV
GET	/admin/logs/auth	†	Logs de autenticación
GET	/admin/logs/auth/suspicious-count	†	Contador de accesos sospechosos
GET	/admin/logs/auth/export	†	Exportar auth log CSV
GET	/admin/settings	†	Obtener configuración
PUT	/admin/settings/{group}	†	Actualizar grupo de config.
POST	/admin/settings/test-email	†	Enviar email de prueba
POST	/admin/cache/clear	†	Vaciar caché Redis

Método	Ruta	Auth	Descripción
POST	/admin/security/block-ip	†	Bloquear IP
GET	/ops/reservations	†	Reservas del día
POST	/ops/reservations/{id}/checkin	†	Check-in
POST	/ops/reservations/{id}/checkout	†	Check-out
POST	/ops/reservations/{id}/items	†	Añadir ítems consumidos
POST	/ops/reservations/{id}/payments	†	Registrar pago
GET	/ops/orders	†	Órdenes activas
POST	/ops/orders/{id}/complete	†	Marcar orden como completa- da
POST	/supervisor/assignments	†	Crear asignación
GET	/supervisor/assignments	†	Listar asignaciones

Apéndice B

Esquema de base de datos

La base de datos de Komorebi Café está definida en 19 archivos de migración SQL ubicados en `migrations/`. La cuadro B.1 describe cada migración.

Tabla B.1: Historial de migraciones SQL

N°	Archivo	Contenido
001	001_infrastructure.sql	Infraestructura base: tablas de configuración
002	002_users_rbac.sql	Usuarios, roles, permisos RBAC
003	003_reviews.sql	Reseñas de clientes
004	004_reservations.sql	Reservas, ítems y time slots
005	005_email_auth.sql	Tokens de email y autenticación
006	006_telegram_bot.sql	Configuración del bot de Telegram
007	007_external_cache.sql	Caché externo de API (festivos, clima)
008	008_animals.sql	Animales, incidentes y galería
009	009_system_settings.sql	Configuración del sistema por grupos
010	010_newsletter.sql	Suscriptores del boletín
011	011_time_slots_waitlist.sql	Franjas horarias y lista de espera
012	012_waitlist.sql	Estructura definitiva de waitlist
012b	012b_reservation_triggers.sql	Triggers SQL de reservas
013	013_loyalty_system.sql	Tarjetas, transacciones y catálogo de fidelización
014	014_staff_shifts.sql	Turnos de personal
015	015_animal_health_checks.sql	Revisiones de salud de animales
016	016_supervisor_assignments.sql	Asignaciones de supervisor
018	018_api_tokens.sql	Tokens Bearer para API REST
—	026--030_*.sql	Correcciones y ajustes adicionales

Apéndice C

Variables de entorno

La configuración de la aplicación sigue el principio 12-Factor III: toda la configuración se externaliza en variables de entorno, nunca en el código fuente. La cuadro C.1 describe las variables principales.

Tabla C.1: Variables de entorno del sistema

Variable	Ejemplo	Descripción
APP_ENV	production	Entorno de ejecución
APP_DEBUG	false	Modo debug (nunca true en prod.)
APP_URL	https://...	URL base pública
DB_HOST	mysql	Host de la base de datos
DB_DATABASE	komorebi	Nombre de la base de datos
DB_USERNAME	komorebi_user	Usuario MySQL
DB_PASSWORD	(secreto)	Contraseña MySQL
REDIS_HOST	redis	Host Redis
REDIS_PORT	6379	Puerto Redis
SESSION_NAME	komorebi_sess	Nombre de la cookie de sesión
SESSION_LIFETIME	7200	Duración en segundos
MAIL_HOST	mailpit	Servidor SMTP
MAIL_PORT	1025	Puerto SMTP
MAIL_FROM	no-reply@...	Remitente de emails
FEATURE_OPS	1	Activa módulo Ops (recepción/cocina)
FEATURE_BACKOFFICE	1	Activa módulo backoffice admin
FEATURE_KEEPER	1	Activa módulo Keeper
TELEGRAM_BOT_TOKEN	(secreto)	Token del bot de Telegram
CLOUDINARY_URL	cloudinary://...	URL de Cloudinary (imágenes)

Apéndice D

Glosario de términos y acrónimos

